2026/2027

FSM

Vision par ordinateur 2

Séance 1 — Introduction et panorama du cours

Niveau 2e année Mastère en informatique

Proposé par Mr Samir Belaid,

Objectifs de la séance

- Resituer les acquis de Vision par ordinateur 1 (ML, DL, CNN, segmentation et détection convolutionnelles).
- Comprendre la logique et la progression du programme de Vision par ordinateur 2 (modèles séquentiels, attention, modèles génératifs, Transformers visuels).
- Situer les briques applicatives émergentes (RAG, agents IA, serveurs MCP, LLM locaux et en ligne) dans le paysage de la recherche actuelle.
- Préparer un travail de présentation sur l'impact de ces technologies sur la démarche de recherche.

PARTIE 1

Rappel de Vision par ordinateur 1

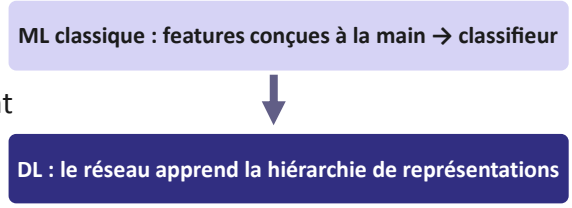
Consolider le socle avant d'aborder les modèles avancés

- Retracer le passage du ML classique au deep learning
- Revoir les trois principes fondateurs du CNN
- Identifier la limite structurante qui motive VPO 2

Du machine learning au deep learning

Partie 1

- **ML classique** : la fonction de décision s'apprend, mais les descripteurs (SIFT, HOG) sont conçus à la main, puis un classifieur (SVM, forêts aléatoires) tranche.
- **Deep learning** : le réseau apprend directement une hiérarchie de représentations à partir des pixels.



À retenir le DL supprime l'ingénierie manuelle des caractéristiques, au prix d'un besoin accru en données et en calcul.

L'architecture CNN : trois principes fondateurs

Partie 1

- **Convolution** : des filtres partagés balayent l'image (peu de paramètres, invariance par translation).
- **Hiérarchie spatiale** : l'empilement de couches augmente le champ récepteur, capturant un contexte de plus en plus large.
- **Sous-échantillonnage (pooling/stride)** : réduit la résolution, apporte une invariance locale, diminue le coût.
- **Modèles de référence** : LeNet, AlexNet, VGG, ResNet (connexions résiduelles).



Pipeline CNN

Convolutions locales, non-linéarité (ReLU) et sous-échantillonnage (pooling) empilés, puis des couches denses pour la décision finale.

Formaliser la convolution

Partie 1

$$y(i,j) = \Sigma_m \Sigma_n x(i+m, j+n) \cdot k(m,n) + b$$

$$W_{out} = (W - F + 2P) / S_{] + 1}$$

- k est le noyau (filtre), b le biais ; W, F, P, S sont la taille d'entrée, du noyau, le *padding* et le *stride*.
- Le champ récepteur croît avec la profondeur : deux couches 3×3 empilées couvrent un champ 5×5 avec moins de paramètres qu'une seule couche 5×5 (argument clé de VGG).

Jalons à connaître

Partie 1

Modèle	Apport clé
LeNet	Premier CNN opérationnel pour la reconnaissance de caractères
AlexNet (2012)	Le tournant : démontre la supériorité du DL sur ImageNet
VGG	Approfondissement systématique de l'architecture
ResNet	Connexions résiduelles : permet d'entraîner des réseaux très profonds
MobileNet / EfficientNet	Familles efficientes, adaptées aux contraintes de ressources

La limite structurante du CNN

Partie 1

- La convolution est un opérateur local : chaque filtre ne "voit" qu'un voisinage restreint.
- Modéliser des dépendances à longue portée exige d'empiler beaucoup de couches.

À retenir c'est l'une des motivations centrales de l'attention et des Transformers, au cœur de la Partie 2.

Détection et segmentation à base de CNN

Partie 1

- **Détection deux étages** : R-CNN → Fast R-CNN → Faster R-CNN (Region Proposal Network).
- **Détection un étage** (temps réel) : YOLO, SSD, RetinaNet (focal loss pour le déséquilibre des classes).
- **Segmentation sémantique** : FCN, U-Net (encodeur-décodeur + connexions de saut), DeepLab (convolutions dilatées).
- **Segmentation par instances** : Mask R-CNN (Faster R-CNN + branche de masque).

À retenir le CNN reste la brique historique de la vision : convolutions locales, partage de poids, hiérarchie spatiale — VPO 2 montrera comment le Transformer remet en cause ce biais de localité.

Métriques d'évaluation à maîtriser

Partie 1

- **Détection** : *intersection sur union* (IoU) entre boîte prédite et boîte vérité, puis précision/rappel à un seuil d'IoU donné.

$$IoU = |Bp \cap Bg| \,/\, |Bp \cup Bg|$$

- AP (aire sous la courbe précision-rappel) et mAP (moyenne sur les classes).
- Segmentation : $mIoU$ (IoU moyen par classe) et $Dice = 2|A \cap B| \,/\, (|A| + |B|)$.

À retenir ces métriques restent valables pour les modèles à base de Transformers : elles permettent une comparaison équitable CNN vs Transformer.

Comparatif des familles CNN

Partie 1

Famille	Principe	Compromis
Deux étages (Faster R-CNN)	Propositions puis classification/régression	Précis, plus lent
Un étage (YOLO, SSD)	Prédiction directe sur une grille	Rapide, temps réel
Segmentation sémantique (U-Net)	Encodeur-décodeur, connexions de saut	Pixels sans distinction d'instance
Segmentation d'instances (Mask R-CNN)	Détection + masque par objet	Plus coûteux

PARTIE 2

Contenu de Vision par ordinateur 2

De la localité convolutive aux mécanismes globaux et génératifs

- Comprendre pourquoi et comment l'attention dépasse la convolution
- Panorama des modèles génératifs (AE, VAE, GAN)
- Découvrir le Vision Transformer et ses usages en détection/segmentation

Modèles récurrents : RNN, LSTM, GRU

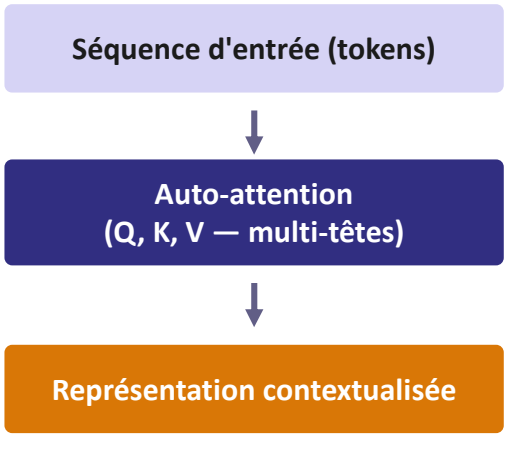
Partie 2

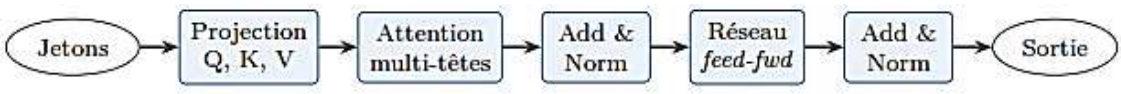
- Conçus pour les séquences (texte, vidéo, séries temporelles) : **l'état caché** propage l'information dans le temps, mais les **RNN** souffrent du *vanishing gradient*.
- **LSTM / GRU** : mécanismes de **portes** régulant la mémoire à long terme et atténue le *vanishing gradient*.
- Ils motivent historiquement le mécanisme d'attention.

Le Transformer et le mécanisme d'attention

Partie 2

- Idée centrale : l'auto-attention permet à chaque élément de pondérer directement tous les autres — dépendances longue portée en une seule couche.
- Composants : requêtes/clés/valeurs (Q, K, V), attention multi-têtes, encodage positionnel, connexions résiduelles.
- C'est l'architecture unificatrice de l'IA moderne : NLP, vision (ViT) et LLM.





Formaliser l'attention

Partie 2

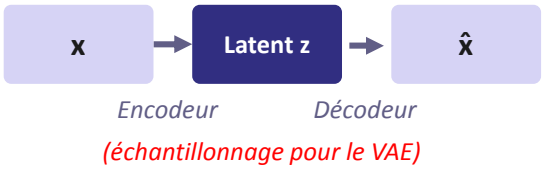
$$Attention(Q,K,V) = softmax(QK^{T} / \sqrt{dk})\,V$$

- d_k : dimension des clés — le facteur $1/\sqrt{dk}$ stabilise les gradients.
- **Attention multi-têtes** : h projections apprises en parallèle, puis concaténées, pour capturer plusieurs types de relations.
- L'opération étant invariante à l'ordre, un encodage positionnel réintroduit l'information de position.
- **Complexité** : $O(n^2 d)$ selon la longueur de séquence n — point sensible pour les images, motivant les variantes efficaces (fenêtres, sparse, linéaire).

Les autoencodeurs (AE)

Partie 2

- Architecture entraînée à reconstruire son entrée.
- **Variantes** : autoencodeur débruiteur, variationnel (VAE, espace latent probabiliste, génératif), masqué (MAE, pré-entraînement auto-supervisé, lié aux ViT).



À retenir apprentissage non/auto-supervisé, réduction de dimension, détection d'anomalies — et fondement conceptuel des modèles génératifs.

Formaliser le VAE

Partie 2

VAE Loss Function$= \mathcal{L}_{reconstruction} + \mathcal{L}_{régularisation}$

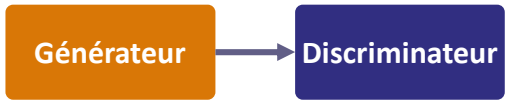
$$\mathcal{L}_{VAE} = Ez_{\sim q(z|x)}[\log p(x|z)] - DKL(\, q(z|x) \,\|\, p(z)\,)$$

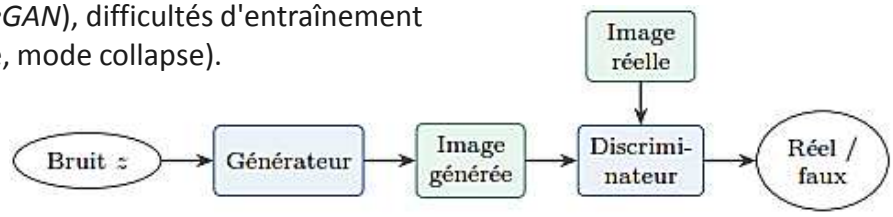
- Un autoencodeur standard minimise l'erreur de reconstruction $\|x - \hat{x}\|^2$.
- Le VAE apprend une distribution latente et maximise cette *borne inférieure de l'évidence* (*ELBO*).
- **Premier terme** : qualité de reconstruction. **Second terme** (D_{KL}) : régularise l'espace latent vers un a priori $p(z)$, souvent gaussien.
- Cette structure probabiliste permet de générer de nouveaux échantillons en tirant z aléatoirement.

Les réseaux antagonistes génératifs (GAN)

Partie 2

- Jeu à deux joueurs : un générateur produit des images, un discriminateur tente de les distinguer du réel.
- Enjeux : réalisme, contrôle (conditional *GAN*, *StyleGAN*), difficultés d'entraînement (instabilité, mode collapse).





À retenir génération et édition d'images, augmentation de données, super-résolution — mais aussi questions éthiques (deepfakes).

Formaliser le GAN

Partie 2

Objectif mathématique

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim P_{data}}[\log D(x)] + \mathbb{E}_{z\sim P_z(z)}[\log(1-D(G(z)))]$$

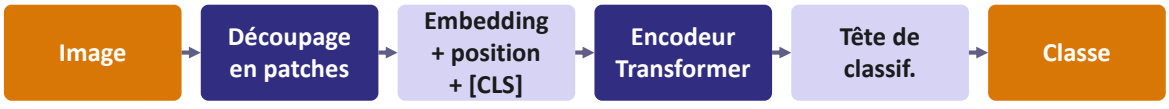
- Jeu minimax entre le générateur G et le discriminateur D. À l'optimum, G reproduit la distribution des données.
- En pratique, l'équilibre est délicat (instabilité, mode collapse) — d'où des variantes (WGAN, gradient penalty).

À retenir **comparaison pédagogique** : les VAE offrent un espace latent structuré mais des images plus floues ; les GAN produisent des images nettes mais plus difficiles à entraîner — deux philosophies génératives complémentaires.

Vision Transformer (ViT)

Partie 2

- **Idée** : découper l'image en patches, les traiter comme une séquence de "tokens" et appliquer un Transformer.
- Compétitif après pré-entraînement massif, il abandonne le biais de localité du CNN
- **Modèles hybrides et efficaces** : *Swin Transformer* (attention par fenêtres hiérarchiques).



L'image devient une séquence de tokens : le Transformer, conçu pour le texte, s'applique alors sans modification structurelle.

Formaliser le ViT : de l'image aux tokens

Partie 2

$$N = H \cdot W \,/\, P^2$$

- Une image $H \times W$ est découpée en N patches de taille $P \times P$, chacun projeté linéairement en un vecteur (*patch embedding*).
- On y ajoute un *encodage positionnel* et un *token de classe* ([*CLS*]).
- La séquence traverse ensuite des blocs Transformer standards.

À retenir sans les biais inductifs du CNN, le ViT n'excède le CNN qu'à partir de grands volumes de données ou avec un pré-entraînement auto-supervisé (*MAE*, *DINO*).

Détection et segmentation à base de Transformers

Partie 2

- **DETR** pose la **détection** comme une prédiction d'ensemble (appariement hongrois), sans ancres ni NMS.
- Ses successeurs (*Deformable DETR*, *Mask2Former*, *SAM*) étendent l'approche à la **segmentation** généraliste.

À retenir on compare explicitement ces méthodes aux approches CNN de la Partie 1 pour mesurer l'apport de l'attention.

Le cas DETR : de la détection à la prédiction d'ensemble

Partie 2

- DETR prédit un ensemble fixe de N objets et établit une correspondance optimale avec la vérité terrain par appariement bipartite (algorithme hongrois).
- Ce coût combine classification et localisation ; il supprime ancres et NMS : le modèle apprend directement "quels objets où".

À retenir l'illustration la plus nette du changement de paradigme entre VPO1 (pipelines heuristiques) et VPO2 (bout en bout, appris).

CNN vs Transformer

Partie 2

Aspect	CNN (VPO1)	Transformer (VPO2)
Portée des dépendances	Locale (champ récepteur)	Globale (attention)
Biais inductif	Fort (localité, translation)	Faible (appris à partir des données)
Besoin en données	Modéré	Élevé (ou pré-entraînement)
Pipeline de détection	Ancres + NMS	Prédiction d'ensemble, bout en bout

Repères chronologiques

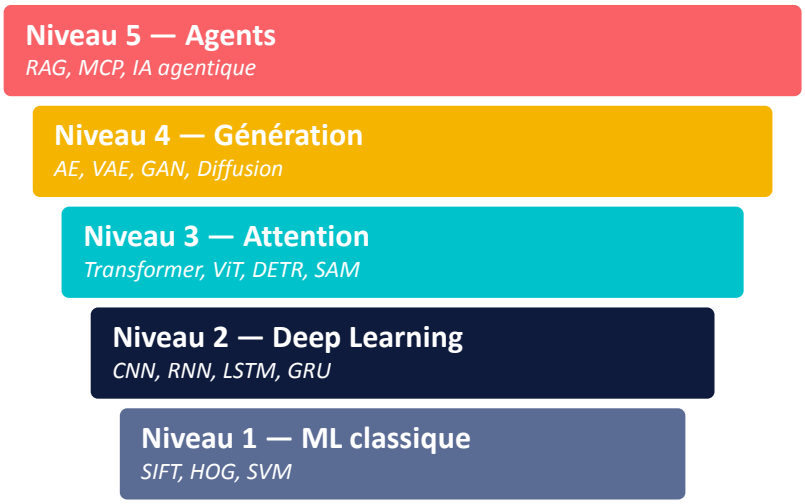
Partie 2

Année	Jalon
1986	Réseaux récurrents (RNN) et rétropropagation dans le temps.
1997	LSTM (Hochreiter & Schmidhuber) : mémoire à long terme par portes.
2006	Autoencodeurs profonds (Hinton & Salakhutdinov) : réduction de dimension apprise.
2014	Année charnière : GRU (Cho), seq2seq (Sutskever), attention (Bahdanau), VAE (Kingma & Welling) et GAN (Goodfellow).
2017	Transformer, « Attention Is All You Need » (Vaswani) : l'attention remplace la récurrence.
2020	DETR (détection par Transformer, Carion) ; DDPM (modèles de diffusion, Ho).
2021	Vision Transformer (ViT, Dosovitskiy) ; Swin Transformer (hiérarchie multi-échelle).
2022	Diffusion latente / Stable Diffusion : génération haute résolution tractable.
2023	Segment Anything (SAM) : segmentation généraliste pilotée par prompt.

Deux ruptures structurent cette histoire : 2014 (essor des modèles génératifs et de l'attention) et 2017 (le Transformer, qui unifie séquence et vision et ouvre la voie à ViT, DETR et à la diffusion).

Repères chronologiques: la pyramide des paradigmes

Partie 2



Synthèse

PARTIE 3

Écosystème applicatif émergent

Une compétence méthodologique pour le chercheur, au-delà d'un simple sujet applicatif

- Distinguer LLM locaux et LLM en ligne, et leurs enjeux pour la recherche
- Comprendre le principe du RAG et son lien avec la vision multimodale
- Découvrir les agents IA, l'IA agentique, le harnais et le protocole MCP

LLM locaux vs LLM en ligne

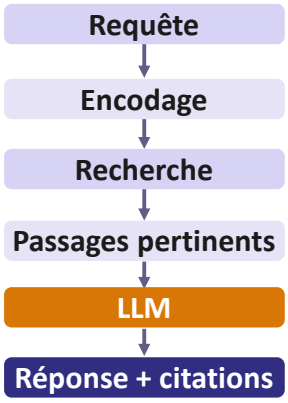
Partie 3

	LLM en ligne (API)	LLM local
Puissance	Maximale, dernière génération	Dépend des ressources matérielles
Infrastructure	Aucune à gérer	À déployer et maintenir
Confidentialité	Dépendance, données transmises	Contrôle total des données
Reproductibilité	Modèle propriétaire, peut évoluer	Version figée, reproductible

RAG (Retrieval-Augmented Generation)

Partie 3

- **Chaîne** : requête → embedding → recherche par similarité dans l'index vectoriel → passages pertinents → concaténation au prompt → LLM → réponse ancrée + citations.
- **Intérêt** : réduire les hallucinations, ancrer les réponses dans des sources vérifiables, intégrer des connaissances à jour sans réentraîner le modèle.
- **Deux verrous de qualité** : la récupération (rappel/précision des passages) et la fidélité de la génération aux sources.



Agents IA

Partie 3

- Un LLM doté d'une capacité à planifier, raisonner et agir en utilisant des outils externes (recherche, code, API).



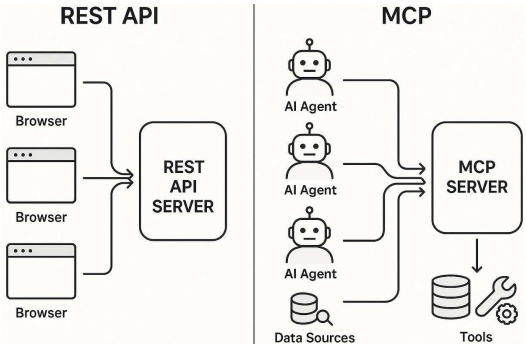
Boucle itérative : percevoir → décider → agir → observer.

À retenir automatisation de tâches complexes (revue de littérature, exécution d'expériences) — mais fiabilité, traçabilité et évaluation restent des questions ouvertes.

Serveur MCP (Model Context Protocol)

Partie 3

- Un protocole standardisé permettant aux modèles/agents de se connecter à des sources de données et outils externes de manière interopérable.
- Normalise l'accès d'un agent aux ressources (fichiers, bases, services) : modularité et réutilisabilité.

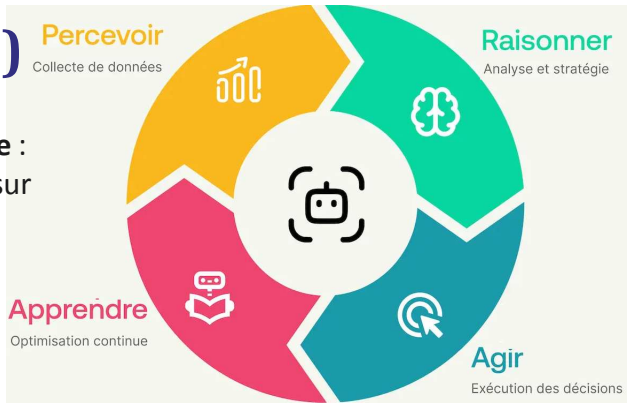


À retenir MCP standardise précisément l'étape "Agir" de la boucle agent : au lieu d'intégrations ad hoc, l'agent découvre et invoque des outils via un protocole commun.

IA agentique (agentic AI)

Partie 3

- **Le modèle ne se contente plus de répondre** : il poursuit un objectif de façon autonome, sur plusieurs étapes, et corrige sa trajectoire.
- **Spectre d'autonomie** : chaînes fixes (workflows, déterministes) → agents (le modèle contrôle lui-même le flux).



- **Briques récurrentes** : *planification*, *mémoire* (court/long terme), *utilisation d'outils*, *réflexion/auto-évaluation*.
- **Architectures multi-agents** : plusieurs agents spécialisés collaborent (planificateur, exécutant, critique).

À retenir passer d'un modèle qui répond à un modèle qui agit et décide.

Harnais (harness)

Partie 3

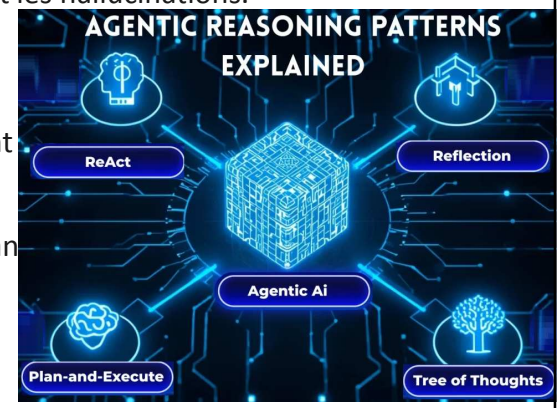
- Un **LLM** "**brut**" ne fait que produire du texte : c'est le **harnais** qui le transforme en **agent** opérationnel.
- Il prend en charge la **boucle de raisonnement**, l'appel effectif des *outils* (via MCP), la gestion *du contexte/mémoire*, et les *garde-fous* (retries, observabilité).
- Il désigne aussi le *harnais d'évaluation* : un cadre standardisé pour comparer des modèles de façon reproductible.
- **Cadres courants** : *LangChain*, *LlamaIndex*, *AutoGPT* et les frameworks d'*agents*.

À retenir sans harnais, un LLM reste un générateur de texte — **enjeux** : *fiabilité*, *coût* (multiplication des appels), *sécurité* (injection de prompt).

Patrons d'agents : ReAct, Reflexion, Planification

Partie 3

- **ReAct** (*Reasoning + Acting*) : alterne pensée → action → observation en boucle ; ancre chaque étape dans un retour concret, réduisant les hallucinations.
- **Reflexion** (*auto-critique*) : après un échec, l'agent rédige une réflexion, la mémorise, puis réessaie — un "apprentissage par renforcement verbal", sans mise à jour des poids.
- **Planification** (*plan-and-execute*) : établit un plan en sous-tâches, puis l'exécute en replanifiant si besoin ; on sépare souvent planificateur et exécuteur.



À retenir ces patrons se combinent : *planifier* une tâche, l'*exécuter* en boucle ReAct, recourir à *Reflexion* en cas d'échec.

Workflow fixe contre **agent autonome**

Partie 3

Critère	Workflow fixe	Agent autonome
Contrôle du flux	Prédéfini par le développeur	Décidé par le modèle à l'exécution
Prédictibilité	Élevée, quasi déterministe	Variable d'une exécution à l'autre
Flexibilité	Faible : chemins figés	Élevée : s'adapte à l'imprévu
Coût et latence	Maîtrisés et stables	Variables (appels multiples)
Débogage	Simple, traçable	Complexe (nécessite observabilité)
Gestion des erreurs	Chemins d'erreur prévus	Auto-correction (retries, Reflexion)
Fiabilité	Élevée sur tâches cadrées	Dépend de la qualité du harnais
Cas d'usage idéal	Tâches répétitives, bien définies	Tâches ouvertes, exploratoires

Repères chronologiques : l'écosystème applicatif

Partie 3

Année	Jalon
2017	Transformer : socle commun des grands modèles de langage
2018	Premiers grands modèles pré-entraînés (BERT, GPT-1)
2020	GPT-3 ; formalisation du RAG (Lewis et al.)
2022	Chaîne de pensée (chain-of-thought) et ReAct ; ChatGPT
2023	GPT-4 ; function calling ; premiers agents autonomes ; naissance des harnais
2024	Essor de l'IA agentique et des systèmes multi-agents ; Model Context Protocol (MCP)
2025	Industrialisation des agents en production : harnais fiabilisés, garde-fous, évaluation reproductible

PARTIE 4

Travail à préparer

Présentations étudiantes

- Choisir un angle (RAG, agents, LLM local vs en ligne, ou MCP) et l'approfondir
- L'illustrer par un cas concret issu de la **littérature** ou d'un usage réel **en recherche**
- Analyser de façon critique **apports**, **limites** et **enjeux éthiques**

Consigne du travail

Partie 4

Préparez une présentation (≈ 15 min + questions) sur : "**Impact des technologies** RAG, agents IA, LLM (locaux/en ligne) et serveurs MCP **sur** l'**avancement des travaux de recherche**."

- Choisir un des angles (RAG, agents, LLM local vs en ligne, ou MCP) et l'approfondir, tout en le situant par rapport aux autres.
- Illustrer par au moins un cas concret issu de la littérature récente ou d'un usage réel en recherche.
- Analyser de façon critique : apports méthodologiques, limites (fiabilité, reproductibilité, biais, hallucinations), enjeux éthiques.
- Proposer une ouverture : en quoi cet outil pourrait transformer votre propre domaine de recherche.